工作周报

> **汇报人：周创钲**
>
> **时间：**3月4日-3月10日

本周工作内容 3月4日-3月10日

重点工作	关键进展	风险及应对措施
写脚本跑通 Binance 数据获取，画出正确的K线图。	• 编写基础连接脚本：使用 ccxt 库初始化 Binance 交易所对象。 • 获取实时行情：编写函数获取特定代币（如 BTC/USDT）的实时价格（Ticker）和 24 小时交易量。 • 获取 K 线历史数据：编写函数调用 fetch_ohlcv 接口，获取指定时间周期（如 1h, 4h, 1d）的 Open, High, Low, Close, Volume 数据。 • 数据清洗与格式化：将获取到的原始列表数据转换为 pandas DataFrame 格式，并正确转换时间戳。	确认 Binance 公开数据接口无需 Key 即可访问，同时申请了一个只读 Key 以防限流

开发CoinGecko 数据获取模块	• 编写 CoinGecko 连接脚本：使用 pycoingecko 库直接调用 API。 • 获取市场概览数据：编写函数获取特定代币的市场排名、当前价格和市值。 • 获取历史价格数据：尝试获取 CoinGecko 的历史价格数据，用于交叉验证。	了解 CoinGecko API 的免费调用频率限制
搭建 FastAPI 后端框架，集成数据获取模块并提供 API 接口。	• 创建项目结构：按 app/api/v1/endpoints/ 分层组织代码，包含 market.py、strategy.py、trading.py、auth.py 等模块。 • 封装市场数据接口：将 Binance/CoinGecko 数据获取逻辑封装为 REST API，供前端调用。 • 配置环境变量：使用 .env 文件管理 API Key、数据库连接等配置，通过 config.py 统一读取。	注意跨域（CORS）配置，确保前端 Next.js 可正常调用后端接口。
搭建 Next.js 前端项目，集成 TradingView 图表组件展示 K 线数据。	· 初始化前端项目：使用 Next.js + TypeScript + Tailwind CSS 创建 frontend/my-app 项目 · 开发 K 线图组件：在 components/charts/TradingViewChart.tsx 中封装 TradingView Lightweight Charts，实现 K 线图渲染 · 开发 Dashboard 页面：在 app/dashboard/page.tsx 中集成行情展示、币种切换、周期选择等功能 · 实现完整的模拟交易功能（下单、持仓、平仓） · 实现 AI Agent 分析面板（支持 3 个 Agent：趋势跟踪、均值回归、风险管理） · 实现 Agent 流式输出展示（支持 Ollama 本地模型的思考链实时展示） · 实现 Agent 胜率计算（基于回测引擎实时计算） · 实现 WebSocket 实时行情连接状态显示 · 实现模拟账户资产展示（初始 10 万 USDT）	WebSocket 连接可能断开，已实现自动重连机制；Ollama 本地服务可能未启动，已添加状态检测和友好提示
开发量化策略技术指标库	· 实现 SMA/EMA 均线计算：支持简单移动平均和指数移动平均，用于判断趋势方向 · 实现布林带（Bollinger Bands）：计算上轨/中轨/下轨及 %B 指标，衡量价格偏离程	指标参数（周期、阈值）需根据具体品种和周期调优，当前使用经典默认值

	度 · 实现 RSI 相对强弱指数：0~100 值域，识别超买（>70）超卖（<30）区域 · 实现 MACD 指标：DIF/DEA/柱状图，金叉买入/死叉卖出 · 实现 ATR 平均真实波幅：用于动态止损和仓位管理	
搭建回测引擎框架	· 设计回测核心流程：数据准备 → 信号生成 → 模拟撮合 → 绩效统计 · 实现资产曲线追踪：记录每根K线结束时的总资产（现金+持仓市值） · 计算关键绩效指标：总收益率、年化收益、最大回撤、夏普比率、胜率、盈亏比 · 支持手续费模拟：单边 0.1% 手续费扣除，贴近真实交易成本 · 支持买入持有基准对比，计算超额收益（Alpha） · 支持参数网格优化，自动搜索最优参数组合 · 支持批量回测，多币种并发回测生成收益排行榜 · 回测结果持久化到 PostgreSQL，支持历史记录查询	回测假设以收盘价成交，实际交易中存在滑点，实盘需预留安全边际
实现均线交叉策略	· 金叉/死叉信号逻辑：短期均线上穿长期均线买入，下穿卖出 · 支持 SMA 和 EMA 两种均线类型 · 提供趋势持仓版本：短均线>长均线时全程持仓 · 回测对比：与买入持有基准对比，计算超额收益（Alpha）	趋势跟踪策略在震荡行情中会产生频繁假信号，建议配合波动率过滤
实现布林带策略	· 均值回归策略：价格触碰下轨买入、触碰上轨卖出 · 收口突破策略：带宽收窄后突破上轨追趋势 · 实时 %B 指标展示：显示当前价格在布林带中的相对位置 · 双策略回测对比：均值回归 vs 趋势突破	强趋势行情中价格可能沿轨道持续运行，均值回归策略会失效
实现 RSI 超买超卖策略	· 经典超买超卖：RSI<30 买入，RSI>70 卖出 · 中轴穿越策略：RSI 上穿/下穿 50 作为趋势确认	RSI 在强趋势中可能长期停留在超买/超卖区，需配合趋势指标使用

	· RSI+布林带组合策略：双重过滤减少假信号 · 三种策略并行回测：输出对比表格便于评估	
数据接入与验证	调用 BinanceService 获取真实历史K线数据 · 支持多周期数据：1d、4h、1h 等 · 所有策略脚本可直接运行，自动拉取数据并输出回测报告 · 验证 API 连接正常：代理配置、API Key 认证通过 · 已实现 ClickHouse 数据缓存层，提升数据查询性能	币安 API 有频率限制，大量回测数据需考虑本地缓存机制
开发模拟交易与风控管理系统	· 全功能撮合引擎： 实现了 PaperTradingService ，支持市价/限价单撮合，内置 0.1% 手续费 与 0.05% 滑点 模拟，真实还原交易损耗。 · 多层级风控体系： 集成 6 大核心风控规则（ RiskManager ），包括 Fat Finger （价格偏离 >5% 拦截）、 最大回撤熔断（>15%）、 日内亏损限额 （>5%）及 Kill Switch 全局开关。 · AI 动态风控： 开发了 RiskAgent ，基于 ATR 指标 实时计算市场波动率，动态输出止损位建议（1x/2x ATR）与风险评级。 · 高性能状态管理： 采用 Redis (TTL 10s) + PostgreSQL 双层架构，确保高并发下账户资金与持仓数据的实时性与持久化。	风险： 限价单撮合目前依赖轮询机制，在极端行情下可能存在成交延迟。 应对： 计划接入 Go Gateway 的 WebSocket 价格流，将轮询改为事件驱动触发撮合。 风险： 实时计算全仓市值（Mark-to-Market）对数据库压力较大。 应对： 已优化为优先读取 Redis 缓存价格，仅在下单关键路径进行数据库强一致性校验。
高性能行情网关开发 (Go + NATS) 构建低延迟行情分发与交易执行核心	· Go Gateway 实现： 基于 Go 语言开发了独立网关 (gateway/)，通过 NATS 消息总线实现高吞吐量的行情数据分发。 · 双向通信： 实现了 market.{symbol}.ticker/kline 的行情发布与 signal.order 的交易指令订阅。 · 交易执行器： 集成 Binance REST API，支持实盘/测试网下单，并内置基础风控检查。	风险： 多语言架构（Go+Python）增加了系统运维复杂度。 应对： 统一使用 Docker Compose 编排，各服务通过 NATS 解耦，互不影响启动顺序。
策略优化引擎升级 实现多模式参数寻优	· 并行网格搜索： 在 optimizer.py 中实现了 GridOptimizer ，利用 ProcessPoolExecutor 支持多核并行回测，大幅提升全参数扫描效率。	风险： 大规模参数寻优对 CPU/内存消耗极大。 应对： 已实现基于 Chunk 的多进程任务分发，未来可引入 Numba

	· 贝叶斯优化集成： 引入 Optuna 库实现 OptunaOptimizer ，支持通过贝叶斯算法高效寻找策略最优参数，减少无效计算。 · 回测引擎重构： 明确拆分向量化回测 (Vectorized) 与事件驱动回测 (EventDriven)，兼顾速度与细节。	进一步加速核心计算逻辑。
多模态 AI Agent 体系 增强大模型分析能力	· 多 LLM 适配： 在原有 Ollama 本地模型基础上，新增 OpenAI 和 OpenRouter 提供商支持 (services/llm/)，提升复杂场景下的推理能力。 · Agent 协同架构： 完善了 Coordinator、Trend、MeanReversion、Risk 四类 Agent 的协作机制，支持流式思考过程输出 (<think> 标签解析)。	风险： 外部 LLM API 调用成本不可控且存在延迟。 应对： 保留 Ollama 本地模型作为低成本/离线备选方案，并实现 Token 使用量监控。
全栈系统容器化与部署 一键式环境搭建	· 微服务编排： 更新 docker-compose.yml ，集成 Frontend (Next.js)、Backend (FastAPI)、Gateway (Go)、PostgreSQL、ClickHouse、Redis 等全套服务。 · 环境配置规范： 完善 .env 和 .env.example ，涵盖 Binance API、NATS 地址、LLM Key 等关键配置。	风险： ClickHouse 等组件资源占用较高，开发机可能卡顿。 应对： 提供精简版启动配置，允许按需开启特定服务（如仅启动核心回测模块）。

本周工作内容 3月13日-3月20日

重点工作	关键进展	风险及应对措施
完善模拟盘交易系统，包括每笔买入、卖出的交易记录，要有相关指标		
引入公认较好的各类型经典量化分析策略，做胜率对比，评价指标清晰化		
使用上述策略进行的每笔模拟盘交易前需经过历史回测数据验证参数适配性		
多币种下策略智能切换		